Uses and applications of the OpenAPI/Swagger specification: a systematic mapping of the literature

Sandra Casas
Instituto de Tecnología Aplicada
Unidad Académica Río Gallegos
Universidad de la Patagonia Austral
Río Gallegos, Argentiina
sicasas@uarg.unpa.edu.ar

Diana Cruz
Instituto de Tecnología Aplicada
Unidad Académica Río Turbio
Universidad de la Patagonia Austral
Río Turbio, Argentiina
dcruz@uart.unpa.edu.ar

Graciela Vidal
Instituto de Tecnología Aplicada
Unidad Académica Río Gallegos
Universidad de la Patagonia Austral
Río Gallegos, Argentiina
gvidal@uarg.unpa.edu.ar

Marcela Constanzo
Instituto de Tecnología Aplicada
Unidad Académica Río Gallegos
Universidad de la Patagonia Austral
Río Gallegos, Argentiina
mconstanzo@uarg.unpa.edu.ar

2021 40th International Conference of the Chilean Computer Science Society (SCCC) | 978-1-6654-0956-8/21/$31.00 ©2021 IEEE | DOI: 10.1109/SCCC54552.2021.9650408

***Abstract*—The OpenAPI or Swagger specification has become the reference choice for describing web APIs (REST, RESTful, etc.). That is why this specification has been and is currently the center of many research articles. Several activities, tasks, and problems associated with software development can be improved, simplified, solved and/or automated with this specification. The purpose of this study is to synthesize the uses and applications of the OpenAPI/Swagger specification. The development activities in which it has been useful, the software artifacts (results) generated from it, and which domains have been covered, among others, are identified. The literature review that we analyzed had a total of 47 articles published between 2011 and 2020. The search corresponded with these parameters: a) mention of the development activities in which the specification has been useful, b) inclusion of the software artifacts (results) that it generates and their corresponding validation, and c) reference to the domains covered. Results show that the OpenAPI/Swagger specification has benefited various software development activities, being the improvement of the documentation the one that predominates (43%). Similarly, 77% of the articles provide solutions to the requirements and/or problems of the web API consumers. The automation of tasks, based on diverse tools, constitutes the most predominant result in 68% of the manuscripts. Besides, 40% of the works consulted cover a specific domain (IoT, Cloud, etc.).**

***Index Terms*—API web, OpenAPI, API Rest, Servicios web**

I. Introducción

En la última década, las APIs web se han convertido en un activo imprescindible para el desarrollo de software de aplicaciones modernas. La industria y las instituciones públicas ofrecen una amplia diversidad de servicios, recursos, datos y funciones, por medio de algún tipo de API web, que son usadas (consumidas) por otras aplicaciones software. De acuerdo con el reporte de RapidAPI[1], de la encuesta a desarrolladores de 120 países, la tendencia a compartir y reutilizar recursos y servicios mediante APIs web crece año a año, el 61,3% de los desarrolladores utilizaron más APIs en 2020 en comparación con 2019 y el 71% de ellos, planean usar incluso más API en 2021 en comparación con 2020. Esto revela que se ha logrado mayor confianza y utilización de las APIs en empresas de todos los tamaños y desarrolladores de una amplia gama de industrias.

Los procesos de producción y uso de APIs web, han presentado nuevos desafíos para el desarrollo de software. Casi desde los inicios un problema, que fue rápidamente reconocido, consistió en la falta de un estándar de especificación de la interfaz de las APIs web. Por un lado, la heterogeneidad dificulta su uso y, por otro lado, el mantenimiento de la documentación en forma sincronizada y actualizada con la implementación real de la API web, es una tarea abrumadora, si es manual. Para cubrir este "gap", aparecieron diversas propuestas de especificación y herramientas para su producción. Hay varias herramientas en competencia para la documentación de API, algunas de ellas se encuentran en un estado bastante maduro, mientras que otras aún están en evolución. Actualmente es posible asegurar que la propuesta que se ha consolidado y se considera un estándar de hecho, es el tándem OpenAPI/Swagger[2].

Swagger es uno de los framework de documentación de API web más populares, usados y aceptados. La primera versión apareció en al año 2011, y en el año 2016, la especificación Swagger pasó a llamarse especificación OpenAPI, cuando la empresa creadora SmartBear, la donó a Open API Initiative. Es por eso, que suele denominarse indistintamente especificación Swagger y/o OpenAPI. Esta proporciona una forma estándar, abierta, portátil, neutral e independiente del lenguaje, de definir los recursos y las operaciones de una API REST, ya sea en JSON o YAML. Este enfoque permite al cliente/consumidor comprender las capacidades del servicio REST sin ningún acceso previo al código de implementación del servicio o la inspección de la red. La especificación OpenAPI/Swagger se ha convertido en la elección de referencia para describir las APIs REST y RESTful.

Como consecuencia de su adopción, la especificación OpenAPI/Swagger ha estado y está en el centro de múltiples iniciativas de investigación. Diversas actividades, tareas y/o problemas asociados al desarrollo de software, pueden verse mejorados, simplificados, resueltos y/o automatizados a partir de la misma. Comúnmente la especificación OpenAPI/Swagger se ha propuesto para realizar pruebas, generar el esqueleto de la

[1]https://rapidapi.com

[2]https://www.openapis.org/ https://swagger.io/

978-1-6654-0956-8/21/$31.00 ©2021 IEEE

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 28,2026 at 01:31:12 UTC from IEEE Xplore. Restrictions apply.

API, de manera automática.

Este estudio de mapeo intenta dimensionar el uso y aplicación de la especificación OpenAPI/Swagger, para el desarrollo de software. Para ello se identifican y analizan los trabajos que han usado y/o aplicado la especificación OpenAPI/Swagger para resolver algún problema asociado al desarrollo de software, más allá del objeto de su creación, la documentación. En otras palabras, nuestros interrogantes son: al disponer de la especificación OpenAPI/Swagger, ¿qué artefactos (resultados) se han creado para resolver o facilitar alguna tarea, actividad o problema del desarrollo? ¿cuáles son los objetivos de estos nuevos artefactos? ¿a qué actividad/fase del desarrollo al que están dirigidos? ¿qué dominios son cubiertos y para que actores (proveedor y/o consumidor)?. En síntesis, qué nuevos aportes han sido impulsados a partir de esta especificación. El significado que damos al término "artefacto" es amplio, consideramos así herramientas, frameworks, métricas, modelos, arquitecturas, procedimientos, entre otros.

Para la ejecución de este trabajo hemos seguido los lineamientos propuestos por [1] [2]. Un total de 47 artículos fueron seleccionados de las principales bases de datos de artículos (ACM, IEEE, Science Direct, Springer), en el periodo 2011-2020. Estos son clasificados por principal fase o actividad del desarrollo a la que dan soporte, tipo de resultado (artefacto), método de validación y dominio de aplicación.

La especificación OpenAPI/Swagger se ha empleado para mejorar diversas actividades del desarrollo de software. Sin embargo, la actividad que más se ha beneficiado resulta ser la misma documentación de la API, ya que a esta corresponden el 43% de los documentos identificados. En cuanto al tipo de resultado, la propuesta de automatización (herramientas) en algún nivel se presenta como herramientas para dar soporte a los enfoques en el 68% de los trabajos y el 77% están dirigidos a consumidores. El método de validación más usado son los ejemplos (47%). Por último, la especificación OpenAPI/Swagger se ha aplicado a dominios tales como IoT, microservicios y computación cloud, entre otros, como se indica en el 40% de los artículos.

La estructura de este documento es como sigue, en la Sección 2, se presentan otros estudios de revisión relacionados, en la Sección 3, se describe la metodología usada; en la Sección 4 se presentan los resultados, y por último en la Sección 5, se expone la discusión y las conclusiones.

II. Trabajos relacionados

Según [3] podría decirse que no existen estudios de revisión que se enfoquen específicamente en las APIs web, aunque otros autores han proporcionado diversos estudios de revisión enfocados a los paradigmas SOA [4] y/o web service [5] con diferentes objetivos, quizás, recientemente se incorporan diversas revisiones en relación con el paradigma de microservicios como [6].

[3] es una revisión sistemática de la literatura de la investigación de APIs publicada entre 2010 y 2018, que comprende 104 artículos. Presenta una descripción general de la investigación de API, sintetiza la investigación del periodo y describe las áreas que necesitan mayor atención. Los hallazgos sugieren que los temas dominantes en la investigación de APIs web pertenecen al diseño y la usabilidad, que hacen foco al dominio tecnológico del esquema de clasificación. El estudio también sugiere que la investigación de API es en gran parte ateórica. La mayoría de los estudios tienden a utilizar experimentos y muestras de código como métodos de investigación frente a los métodos cualitativos y cuantitativos dominantes. El estudio recomienda que la investigación de API también debe considerarse desde una perspectiva sociotécnica.

En [7] aunque no se trata de un estudio de revisión clásico, ya que el objetivo de los autores es construir un conjunto integral de heurísticas y pautas para la usabilidad de API, efectúa una síntesis estructurada de la literatura existente sobre usabilidad de API. Este conjunto lo conectan con un modelo de usabilidad, lo que permite comprobar si están abordando problemas reales de usabilidad. Sus conclusiones son que la literatura existente sobre la usabilidad de las APIs es heterogénea en términos de objetivos, alcance y audiencia; y su conexión con las definiciones aceptadas de usabilidad rara vez se hace explícita. El uso de métricas para medir la usabilidad de la API se enfoca solo en características medibles excluyendo aquellos aspectos de usabilidad que están relacionados con la subjetividad de las opiniones. El estudio no es específico de APIs web, ya que incluye mayormente estudios sobre APIs locales.

En [8] presentan un estado del arte en relación con la evolución de las APIs web, como resultado definen un framework de clasificación que considera tanto los cambios que pueden ocurrir en las API como las razones detrás de ellos.

En [9] se presenta un mapeo sistemático sobre herramientas y enfoques de generación de documentación API analizados y categorizados de diversas formas. Entre otras cosas, el informe presenta una descripción general de qué tipo de herramientas se han desarrollado, qué tipo de documentación generan y qué fuentes requieren los enfoques de documentación. Se identifican 42 estudios del periodo 2000-2016, en el cual no se distingue si abarca todo tipo de API (locales y web). Sobre el que existe otra versión en [10].

En [11] se informa un estudio de mapeo sistemático que consta de 21 estudios en el cual los autores capturan qué conocimientos o artefactos deben incluirse en la documentación de la API; y una taxonomía estructurada, basada en las recomendaciones consolidadas de estos estudios.

En [12] se presenta una revisión sistemática de la literatura, que tiene el objetivo de recopilar prácticas y capacidades de gestión de API, así como proponer una definición integral del tema. En el ámbito de este trabajo, una práctica se define como cualquier práctica que tiene el objetivo expreso de mejorar, fomentar y gestionar el uso de API. Las capacidades se definen como la capacidad de lograr un objetivo determinado relacionado con API Management, mediante la ejecución de dos o más prácticas interrelacionadas. Los autores identifican 42 artículos en el periodo 2011-2019, sobre los cuales logran recopilar 24 definiciones únicas para el tema, 114 prácticas y 39 capacidades.

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 28,2026 at 01:31:12 UTC from IEEE Xplore. Restrictions apply.

III. METODOLOGÍA

A continuación, se detalla el plan de mapeo sistemático que se ha ejecutado, el mismo se basa en lo presentado en [1]. Este estudio se realizó durante el primer semestre del 2021.

El objetivo de este estudio es identificar, analizar y sintetizar los trabajos publicados durante 2011 y 2020 que usen o apliquen la especificación OpenAPI/Swagger para el desarrollo de software. Para cumplir el objetivo, se han formulado las siguientes preguntas de investigación:

P1: ¿Qué actividades del desarrollo de software se proponen mejorar a partir del uso / aplicación de la especificación OpenAPI/Swagger?

P2: ¿Qué tipo de artefactos es el resultado del estudio? ¿Cómo han sido validados?

P3: ¿En qué dominio los trabajos han sido dirigidos / aplicados?

En cuanto a la estrategia de búsqueda para encontrar todos los estudios primarios relevantes que respondan a las preguntas de investigación, se define la cadena de búsqueda y se seleccionan las bases de datos relevantes. Luego, se definen los criterios y el procedimiento de inclusión y exclusión de los estudios. Las palabras clave identificadas a partir de las preguntas de investigación se agruparon para formar la siguiente cadena de búsqueda:

{"swagger" OR "openapi" OR {"web api" AND "specification"} OR {"rest api" AND "specification"} OR {"restful api" AND "specification"}}

Las búsquedas se realizaron en las bases de datos de IEEE Xplore, ACM, Science Direct y Springer, que han sido seleccionadas por contener estudios con revisión de pares. Dada las características de cada base de datos, las consultas se realizaron de la siguiente forma: en IEEE se ha especificado la cadena en todos los términos; en ACM y Science Direct se buscó por título, resumen y palabras clave; mientras que en Springer como contenido del título. En la búsqueda se incluyeron los artículos publicados entre 2011 y 2020.

La herramienta Zotero se utilizó para obtener los registros completos de los estudios obtenidos en las búsquedas y JabRef se utilizó para eliminar duplicados entre los resultados de las distintas bases de datos.

Los estudios obtenidos se ordenaron por autores y se repartió equitativamente entre las cuatro autoras para la selección de estudios, con el objetivo de facilitar el descarte de aquellos trabajos que son complementarios de un grupo de investigación y que se presentan en distintos eventos científicos. Siguiendo con este esquema, se excluyeron los artículos basados en títulos y resúmenes, así como en la lectura de texto completo y la evaluación de la calidad. La lectura de análisis se realizó en tres fases: 1) Todas las autoras aplicaron los criterios de inclusión y exclusión a cada estudio asignado para su análisis, donde se clasificó por actividad o etapa de desarrollo y dimensión (consumidor o proveedor) del aporte que realizan; 2) Del resultado de la fase 1 de selección de estudios, se reordenó los estudios por etapa de desarrollo y se asignó a cada autora un grupo de etapas para un segundo nivel de análisis. Se leyó a texto completo y se aplicó los criterios de calidad; 3) Se extrajeron todos los datos para la síntesis por temas.

Los criterios de inclusión (I) y exclusión (E) a títulos y resúmenes fueron:

(I) El estudio aborda la generación y/o mejora artefactos para el desarrollo de software que consume o produce APIs web; y generalmente tiene como objetivo mejorar las tareas de desarrollo de software aprovechando la estructura / aplicación de la especificación;

(I) El estudio tiene como objetivo general y/o final mejorar la calidad y/o productividad del desarrollo de software en el que intervienen APIS web;

(E) El estudio no está escrito en inglés;

(E) No se puede acceder al estudio en texto completo;

(E) El estudio es un duplicado de otro estudio;

(E) El estudio usa la especificación solo como medio de documentación;

(E) El estudio es una tesis o literatura gris.

La evaluación de la calidad implicó que cada artículo 1) expresara claramente el problema y/u objetivo del trabajo; 2) presentara un resultado; 3) presentara una validación. Para clasificar y/o categorizar los tipos de resultados y los tipos de validación se tomó como framework la propuesta de Shaw [13]. Shaw identifica los siguientes tipos de resultados, procedimiento o técnica, método descriptivo o cualitativo, modelo empírico, modelo analítico, notación o herramienta, solución específica o prototipo, respuesta o justificación, y reporte de experiencia. En cuanto a los tipos de validación identifica los siguientes, análisis, evaluación, ejemplo, experiencia, persuasión, y opinión.

El formulario de extracción de datos tiene las siguientes secciones que permiten contestar las preguntas de investigación: título, autores, revista / conferencia, año, dimensión (proveedor / consumidor), problema / objetivo / pregunta de investigación, actividad / etapa de desarrollo de software para la que proponen, resultado y validación.

Al ejecutar las búsquedas en cada base de datos resultaron 26 en ACM, 66 en IEEE, 12 en Science Direct y 50 en Springer. Con JabRef se encontraron 5 artículos publicados de forma duplicada en las bases de datos, resultando 149 artículos del conjunto inicial de búsqueda. Luego, al aplicar los criterios de exclusión se descartaron 102 artículos, cuyo detalle se muestra en la Tabla I. Finalmente, la cantidad de estudios para el análisis con lectura del artículo completo resultaron 47. Los estudios analizados en mapeo se detallan en el apéndice.

IV. RESULTADOS

En esta Sección se presentan los hallazgos de la ejecución del mapeo sistemático, los cuales corresponden a la selección de documentos enunciados en el apéndice de referencias de estudios.

La Tabla II presenta la distribución por año de los artículos que fueron analizados. En cuanto al tipo de artículo, su

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 28,2026 at 01:31:12 UTC from IEEE Xplore. Restrictions apply.

TABLA I
EXCLUSIÓN E INCLUSIÓN DE ESTUDIOS

Criterio	Cantidad
Resultado de búsqueda inicial	149
Descartados porque no pertenecen al tema	71
Descartados por duplicados	8
Descartados por falta de texto completo	15
Descartados por no pasar la evaluación de calidad	8
Total de analizados para los resultados	**47**

distribución fue: 23 conferencias, 22 capítulos de libro y dos artículos.

TABLA II
DISTRIBUCIÓN POR AÑO DE PUBLICACIÓN

2011	2012	2013	2014	2015	2016	2017	2018	2019	2020
1	1	0	0	0	3	12	8	8	14

A continuación, se responden los interrogantes planteados anteriormente. En primer lugar, ante la pregunta P1 ¿Qué actividades del desarrollo de software se proponen mejorar a partir del uso / aplicación de la especificación OpenAPI / Swagger? Se identifican 10 actividades del desarrollo de software, en las cuales se usa o aplica la especificación OpenAPI en alguna forma, para mejorar, facilitar o automatizar actividades o tareas, como se muestra en la Tabla III. También se han distinguido si estas propuestas están dirigidas al proveedor o al consumidor de la API web.

TABLA III
ACTIVIDADES/FASES POR DIMENSIÓN PROVEEDOR/CONSUMIDOR

Actividad/Fase	Proveedor	Consumidor	Cantidad por A/F
Documentación de API	A3, A6, A7, A10, A11, A12, A16, A19, A21	A1, A2, A3, A5, A6, A7, A8, A9, A10, A11, A12, A13, A14, A15, A17, A18, A20, A21	20
Descubrimiento		A29, A39, A41, A44	4
Composición		A4, A35, A39, A40	4
Implementación		A36, A38, A40, A43, A45	5
Pruebas	A22, A23, A26	A24, A25, A27, A28	7
Gobierno	A32	A32	1
Modelado	A7, A37		2
RNF	A34, A47	A31	3
Evolución	A30, A37	A37	2
Desarrollo de API	A33, A37, A42, A46		4
Cantidad por dimensión	20	36	

En la categoría Implementación refiere a propuestas enfocadas en las aplicaciones consumidoras o clientes, a diferencia de la categoría Desarrollo de API. Con la sigla RNF, se han agrupado trabajos relacionados a requerimientos no funcionales.

En cuanto a la pregunta P2, ¿Qué tipo de artefactos es el resultado del estudio? ¿Cómo han sido validados?, se muestra en la Tabla IV, los artefactos que producen estos estudios resultando ser en la mayoría herramientas, pero también procedimientos y/o técnicas. Por otro lado, el tipo de validación utilizada en los estudios se muestra en la Tabla V.

TABLA IV
TIPO DE RESULTADO

Resultado	Estudios	Cantidad
Herramienta	A1, A2, A3, A4, A5, A6, A8, A9, A10, A11, A15, A16, A17, A18, A22, A23, A24, A25, A26, A27, A31, A32, A33, A35, A36, A37, A40, A42, A43, A45, A46, A47	32
Procedimiento / Técnica	A2, A6, A7, A9, A12, A13, A16, A19, A21, A22, A23, A24, A25, A26, A27, A29, A30, A33, A35, A37, A39, A40, A41, A43, A44, A47	26
Modelo cualitativo / descriptivo	A1, A4, A20, A28, A31, A33, A36, A37, A38, A44, A45	11
Solución Especifica	A14, A36, A38	3
Modelo Empirico	A7, A34	2

TABLA V
TIPO DE VALIDACIÓN

Validación	Estudios	Cantidad
Evaluación	A11, A18, A31, A34, A36, A40, A41, A43, A45	9
Ejemplos	A3, A4, A5, A7, A8, A10, A12, A13, A16, A17, A24, A26, A27, A33, A35, A37, A39, A41, A42, A44, A46, A47	22
Experiencia	A1, A2, A6, A9, A14, A15, A19, A20, A21, A22, A23, A25, A29, A30, A32, A38, A45	17
Análisis	A28	1

TABLA VI
DOMINIO DE APLICACIÓN

Dominio	Estudios	Cantidad
IoT	A14, A31, A35, A42, A46	5
Cloud	A8, A24, A27, A31	4
Microservicios	A26, A29, A30, A32, A34	5
Mashup	A39, A43, A45	3
Móvil	A42	1
Redes de sensores	A38	1
Mineria de datos	A9, A36	2

Finalmente, la Tabla VI responde a la pregunta P3 ¿En qué dominio los trabajos han sido dirigidos / aplicados? Aunque los dominios de aplicación marcan la tendencia de los aportes de estudio, tal como IoT, cloud, microservicios, mashup, móvil, redes de sensores y minería de datos, la mayoría de los estudios intentan presentar soluciones generalizadas.

A continuación, se presenta una síntesis de los trabajos agrupados por los temas que se han identificado.

A. Documentación

Los trabajos incluidos en el ítem documentación han sido sub-clasificados en tres grupos, según su propósito, extensión de la especificación OpenAPI, generación de la especificación y otros artefactos o herramientas. Los trabajos que extienden la

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 28,2026 at 01:31:12 UTC from IEEE Xplore. Restrictions apply.

especificaron OpenAPI, son aquellos que proponen la incorporación de descripciones semánticas, anotaciones y metadatos a la definición OpenAPI original, [A17][A19][A2][A1] con el fin de incluir información sobre un dominio específico o aclarar posibles discrepancias. La extensión también puede incluir la definición de dependencias entre parámetros [A18] y restricciones de parámetros [A21]. Los atributos incorporados representan un valor agregado a la documentación existente, mejorando su fiabilidad y comprensión. En [A20] se mejora la descripción semántica específicamente para facilitar la composición en modelos y soluciones de software. En el grupo de generación automática los trabajos presentan mecanismos [A6] y nuevos modelos [A7] que captan las características del modelo OpenAPI, para luego, hacer uso de técnicas de lenguaje natural, ontologías [A10] [A13] y/o plantillas NLG (Generadores de Lenguaje Natural) [A12] y generar automática o semi-automáticamente especificaciones OpenAPI o compatibles, mejoradas y adaptadas a las necesidades particulares de negocios y servicios. En su mayoría están destinados proveedores y consumidores de API web. En la categoría otros artefactos o herramientas se incluyen propuestas de frameworks, herramientas, modelos, procedimientos, reportes, entre otros, que están basados o que utilizan como entrada las especificaciones OpenAPI y se aplican para generar o producir un artefacto complementario a la especificación. Estos artefactos trabajan de manera individual o integrada con otros (ecosistemas), automatizando tareas en diversas etapas del proceso de desarrollo, tanto para el proveedor, consumidor o ambos. Las herramientas presentan diferentes funciones, entre ellas crear, visualizar y administrar especificaciones OpenAPI [A3] que incluyen SLA, además generar y diseñar Interfaces [A14] [A15] [A9] [A16] o proveer un chatBoot generado a partir de especificaciones OpenAPI [A5] [A8] que ayudan a los consumidores en detalles de diseño e implementación.

B. Descubrimiento

El descubrimiento de servicios y/o recursos es un proceso que tiene como finalidad encontrar un Servicio Web apropiado que cumpla con los requerimientos que proporciona el solicitante. La especificación OpenAPI/Swagger, también ha sido utilizada para mejorar este proceso, por lo general se utiliza como entrada, y luego mediante procesos de emparejamiento, comparación y análisis automatizados se realiza el proceso. En [A29] proponen un proceso automatizado para identificar microservicios candidatos mediante un análisis de similitud semántico y ligero, independiente del dominio. A partir de la descripción de conceptos y funcionalidad disponible en la especificación OpenAPI de entrada se analiza con respecto a un vocabulario de referencia. En [A41] se presenta un enfoque que tiene como objetivo identificar las transiciones entre los recursos de una API mediante el análisis sistemático de una descripción de servicio OpenAPI. A partir de la especificación, se realizan una serie de pasos, que convierten las URIS, en plantillas, modelos y finalmente un cliente proxi, que actúa como intermediario entre los clientes y las APIs. El proxy crea e inyecta los hipervínculos correspondientes, que representan las transiciones identificadas, en la carga útil de respuesta tan pronto como un cliente solicita un recurso. Todos los pasos desde el análisis hasta la generación del proxy utilizan la descripción del servicio como la única entrada externa. El resultado final es un soporte de navegación similar a HATEOAS a través de una API, que libera al cliente de conocer y crear URI. En [A44] se propone el descubrimiento de servicios (APIs) a partir de sus posibilidades de composición. El enfoque TAD (Transformation-Annotation-Discovery), transforma los documentos OpenAPI /Swagger de los servicios RESTful en una estructura del grafo que incluyen conceptos semánticos, luego se lleva a cabo la composición del servicio según los requisitos del usuario. A partir de diversas técnicas, se automatiza la verificación de la compatibilidad entre los requisitos del usuario y los servicios descubiertos. En [A39] proponen un framework de mashup basado en QoS y un repositorio para almacenar las especificaciones OpenAPI/Swagger, las cuales son analizadas a partir de métricas de QoS, que sirven para la selección en tiempo de ejecución de estas. Se identifica y clasifica las APIs (a partir de la especificación) con funcionalidad similar.

C. Composición

La composición de servicios es el proceso mediante el cual dos o más servicios que ofrecen una funcionalidad básica se combinan para ofrecer una más compleja. La especificación OpenAPI/Swagger se ha utilizado para facilitar el proceso manual, tedioso y propenso a errores que es la creación de las aplicaciones que consumen las APIs, ya sea el descubrimiento, también los procesos de composición e integración. En [A4] para facilitar este proceso, proponen un enfoque ligero, dado un conjunto inicial de definiciones de OpenAPI. Como resultado de la composición, obtienen una API global que oculta la complejidad del proceso de composición al usuario. Para facilitar el consumo de la API global, la exponen como un servicio OData, que permite crear recursos que se definen de acuerdo con un modelo de datos de entidad (EDM) y pueden ser consultados por clientes web utilizando un lenguaje de consulta basado en URL. El EDM corresponde al esquema de datos detrás de la API global, que se genera durante el proceso de composición basado en el descubrimiento de coincidencias entre los datos individuales. Presentan una herramienta en Java y Eclipse. El trabajo [A35] presenta la composición dinámica de aplicaciones web descriptas utilizando especificaciones OpenAPI con aplicaciones de IoT. La solución se basa en la plataforma de IoT llamada LiquidIoT. En esencia se presenta un nuevo enfoque para la orquestación dinámica de aplicaciones IoT y aplicaciones web. Las especificaciones OpenApi son utilizadas en la etapa inicial del proceso. [A39] es también composición. [A40] que se describe más adelanta también da facilidades para la composición.

D. Implementación

En la categoría implementación se han incluido los documentos que buscan mejorar algún aspecto del desarrollo, principalmente como facilitar, agilizar y simplificar las solicitudes,

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 28,2026 at 01:31:12 UTC from IEEE Xplore. Restrictions apply.

consultas, mensajes y llamadas a las APIs web, las cuales pueden ser muy heterogéneas y/o complejas. Se utilizan y/o crean distintos mecanismos a partir del procesamiento de la especificación OpenAPI. En [A38] presenta una arquitectura de acceso a servicios unificados, interfaces y formatos de mensajes para redes de sensores inalámbricos (WSN) denominada colectivamente WSN OpenAPI. En [A36] se presenta el framework RestMule, para manejar varias políticas de servicio de manera transparente. Como resultado, los clientes de RestMule generados a partir de especificaciones de OpenAPI son adecuados para escenarios intensivos de obtención de datos. En [A43] se propone un enrutador de mensajes basado en una interfaz de OpenAPI para el desarrollo de servicios mashup. El enrutador de mensajes apoya el desarrollo del servicio mashup al proporcionar funciones útiles requeridas por el desarrollador en cooperación con varias plataformas de acuerdo con los mensajes de solicitud transmitidos a través de OpenAPI. En [A40] se propone manejar las peticiones a las APIs REST como consultas GraphQL. La solución basada en envoltorios (wrappers), que se generan automáticamente para las APIs, a partir de la especificación OpenAPI.

E. Evolución

En [A30] se propone un enfoque para recopilar métricas de mantenibilidad para sistemas basados en servicios y microservicios, aplicando análisis estático sobre la especificación OpenAPI/Swagger. Así un conjunto de 10 métricas se calcula automáticamente. El conjunto se compone de siete métricas de complejidad, una de tamaño y dos de cohesión.

F. Desarrollo de APIs

La asistencia con herramientas para el desarrollo de las APIs facilita la tarea de los proveedores, desde enfoques dirigidos por modelos hasta frameworks basados en ontologías. En [A37] se propone un enfoque automatizado que a partir de un metamodelo de representación de servicios web y la especificación OpenAPI permite la creación rápida de prototipos, la verificación del modelo, la compatibilidad con lenguajes de programación comunes y la alineación entre la documentación y la implementación. También herramientas para implementar y desplegar APIs en contextos específicos, como APIGEND [A42] para Android e IoT o la herramienta WoTDL2API [A46] que genera automáticamente una API RESTful en ejecución basada en la especificación OpenAPI y proporcionar interoperabilidad entre los dispositivos de IoT. En [A33] proponen un framework que a partir de ontologías permite crear automáticamente APIs REST mantenibles con documentación, pruebas unitarias y validación automatizada.

G. Pruebas

Una de las aplicaciones típicas de la especificación OpenAPI, refiere a posibilidad de generar automáticamente casos de prueba. Se encuentran los frameworks para generación pruebas dinámicas de APIs REST, RESTest [A22], RESTESTGEN [A26], AT Cloud [A27] y un enfoque de metamodelos propuesto por [A23] para realizar pruebas funcionales; mientras que RESTler [A24] se centra en pruebas no funcionales. Estos trabajos utilizan estrategias de caja negra tomando como entrada la especificación OpenApi para la automatización de generación de casos de pruebas usando diversas técnicas de testing (basada en restricciones, random, fuzzing, etc.) También existen aplicaciones para casos de pruebas basadas en análisis estático, apuntan a las aplicaciones clientes desarrollados en JavaScript, como los trabajos [A28] y [A25]. En ambos trabajos verifican la consistencia de las solicitudes de servicios de las aplicaciones con respecto al servicio ofrecido por las API web.

H. Requerimientos No Funcionales

Algunos artículos se enfocan en requerimientos no funcionales (rendimiento, seguridad y calidad). En [A34] proponen un enfoque para comparar aplicaciones completas de microservicios basadas en REST, aplicando patrones de interacción abstractos y reutilizables. En tiempo de ejecución y a partir de una descripción de interfaz como la especificación OpenAPI resuelven y vincular automáticamente los patrones de carga de trabajo al punto final concreto antes de ejecutar el punto de referencia y recopilar resultados, de esta forma logran especialmente la capacidad de resolver dependencias de datos complejas en los puntos finales de microservicios. En [A31] proponen un enfoque de protección a nivel de red, basado en la definición y validación de contratos, los cuales son modelados a partir de una especificación Swagger. Los contratos permiten inspeccionar los mensajes entre la cloud de IoT y las puertas de enlace del cliente, permite auditar fácilmente el alcance de los términos, valida automáticamente las solicitudes/respuestas de la API, detectar y detener violaciones. También se aporta una arquitectura de verificación de mensajes de IoT que, utilizando modelos de descripción de OpenAPI, que permite crear y hacer cumplir contratos de mensajes entre el proveedor de soluciones de IoT y su cliente. En [A47] se presenta un framework que permite un análisis estructural de la API, basado en las especificaciones, que realiza un análisis integral y bien estructurado, para demostrar que una API es completamente compatible con REST. El análisis permite la creación de un inventario de la API que muestra el estado actual de la misma. El inventario contiene métricas y una representación gráfica, permitiendo identificar principales características de cada API y sus deficiencias respecto al estilo arquitectónico REST.

I. Gobierno

En [A32] para mejorar el gobierno de APIs presentan un conjunto de herramientas (ecosistema) basado en acuerdos a nivel de servicio (SLA) que mejoran el ciclo de vida de APIs. Estas herramientas son, un editor SLA, una biblioteca de instrumentación, un Gateway, entre otras. La especificación OpenAPI se usa como entrada del editor SLA.

V. Discusión y conclusiones

a) Amenazas de la validez: Las amenazas a la validez y sesgos se han minimizado a partir de las siguientes acciones,

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 28,2026 at 01:31:12 UTC from IEEE Xplore. Restrictions apply.

1) dado que el objeto de estudio, la especificación OpenAPI/Swagger está directamente relacionada a las APIs web, y estas cruzan tanto áreas de la ingeniería de software pero también nichos de infraestructura tecnológica (IoT, microservicios, etc.), los términos y combinaciones de las cadenas de búsqueda se probaron y ajustaron varias veces en diversas bases de datos para recuperar los documentos más apropiados, 2) la validez descriptiva se controló a partir del diseño de un formulario de recopilación de datos para respaldar el registro de los mismos. Este formulario se estructuró de manera tal, que permitió responder las preguntas y atender los criterios de calidad, 3) la validez teórica se controló de la siguiente manera, todos los abstract fueron leídos por al menos dos autores, los formularios de extracción de datos e informes de cada artículo fueron revisados al menos por dos autores, una de ellas por la primera autora. Para minimizar los sesgos de interpretación, se aplicaron clasificaciones de resultados y validaciones conocidos [13].

b) Conclusiones: Existen algunos estudios de revisión con relación a APIs web. Sin embargo, no se han hallado estudios enfocados en analizar sus especificaciones en algún sentido. Desde este punto de vista, este trabajo es original. Otra cuestión es que los estudios de revisión de APIs suelen incluir y abarcar a las APIs locales. La investigación de APIs locales antecede y supera al de APIs web, lo cual marca en cierta medida los resultados finales. Hacen falta más revisiones sobre APIs web en particular. Observamos que algunas de las actividades identificadas en este estudio de mapeo coinciden con los temas presentados en [3].

La especificación OpenAPI/Swagger se ha empleado para mejorar diversas actividades del desarrollo como el descubrimiento y composición de servicios, la generación automática de la API, dar soporte al gobierno de APIs, y otras actividades relacionadas al consumo en forma aisladas (implementación y/o prueba). Sin embargo, la actividad que más se ha beneficiado resulta ser la misma documentación de la API, ya que a esta corresponden el 43% de los documentos identificados. En cuanto al tipo de resultado (artefacto), la preocupación por liberar a los desarrolladores de las tareas manuales queda manifiesta, en tanto la propuesta de automatización (herramientas) en algún nivel, que se presenta como herramientas para dar soporte a los enfoques en el 68% de los trabajos. No obstante, la propuesta de procedimientos y técnicas, también se destaca 55%. El método de validación más usado son los ejemplos (47%). La especificación OpenAPI/Swagger se ha aplicado a dominios tales como IoT, microservicios y computación cloud, entre otros, como se indica en el 40% de los artículos. Desde nuestra interpretación, estos valores indican cierta tendencia, y los vacíos que existen. Una consideración es que varios trabajos aplican a más de una categoría, en tanto soportan distintas actividades, o proponen más de un tipo de resultados, o aplican a varios dominio y actores. Los tipos de resultados y método de validaciones preponderantes indican, al igual que en [3], que los estudios se caracterizan por una orientación tecnológica y ateórica.

c) Trabajo futuro: Por tratarse de un estudio de mapeo, resulta necesario ahondar de manera más exhaustiva la revisión, pero planteando objetivos más específicos. En este sentido, nuestras próximas investigaciones se centrarán en los tópicos de pruebas y métricas de APIs web.

REFERENCIAS

[1] K. Petersen, R. Feldt, S. Mujtaba, and M. Mattsson, "Systematic Mapping Studies in Software Engineering." in EASE, 2008, pp. 68–77.

[2] B. Kitchenham, O. P. Brereton, D. Budgen, M. Turner, J. Bailey, and S. Linkman. Systematic literature reviews in software engineering: A systematic literature review. Information and Software Technology, 51(1):7-15, 2009.

[3] R. Boateng, J. Ofoeda, and J. Effah, "Application Programming Interface (API) Research," Int. J. Enterp. Inf. Syst., vol. 15, no. 3, pp. 76–95, Jul. 2019, doi: 10.4018/IJEIS.2019070105.

[4] S. Pulparambil and Y. Baghdadi, "Service oriented architecture maturity models: A systematic literature review," Comput. Stand. Interfaces, vol. 61, pp. 65–76, Jan. 2019, doi: 10.1016/J.CSI.2018.05.001.

[5] E. A. Setyawan and F. Hidayat, "Web Services Security and Threats: A Systematic Literature Review," 2020 International Conference on ICT for Smart Society (ICISS), 2020, pp. 1-6, doi: 10.1109/ICISS50791.2020.9307569.

[6] A. Pereira-Vale, E. B. Fernandez, R. Monge, H. Astudillo, and G. Márquez, "Security in microservice-based systems: A Multivocal literature review," Comput. Secur., vol. 103, p. 102200, Apr. 2021, doi: 10.1016/J.COSE.2021.102200.

[7] E. Mosqueira-Rey, D. Alonso-Ríos, V. Moret-Bonillo, I. Fernández-Varela, and D. Álvarez-Estévez, "A systematic approach to API usability: Taxonomy-derived criteria and a case study," Inf. Softw. Technol., vol. 97, pp. 46–63, May 2018, doi: 10.1016/J.INFSOF.2017.12.010.

[8] R. Koçi, X. Franch, P. Jovanovic and A. Abelló, "Classification of Changes in API Evolution," 2019 IEEE 23rd International Enterprise Distributed Object Computing Conference (EDOC), 2019, pp. 243-249, doi: 10.1109/EDOC.2019.00037

[9] Nybom, K., Ashraf, A., & Porres Paltor, I.. A Systematic Mapping Study on Tools for API Documentation Generation. Turku Centre for Computer Science, 2017, doi: 10.13140/RG.2.2.31115.49444

[10] K. Nybom, A. Ashraf and I. Porres, "A Systematic Mapping Study on API Documentation Generation Approaches," 44th Euromicro Conference on Software Engineering and Advanced Applications (SEAA), 2018, pp. 462-469, doi: 10.1109/SEAA.2018.00081.

[11] A. Cummaudo, R. Vasa and J. Grundy, "What should I document? A preliminary systematic mapping study into API documentation knowledge," ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM), 2019, pp. 1-6, doi: 10.1109/ESEM.2019.8870148.

[12] M. Mathijssen, M. Overeem, and S. Jansen, "Identification of Practices and Capabilities in API Management: A Systematic Literature Review," Jun. 2020, Accessed: Aug. 08, 2021. [Online]. Available: https://arxiv.org/abs/2006.10481v1.

[13] M. Shaw, "Writing good software engineering research papers: minitutorial," In Proceedings of the 25th International Conference on Software Engineering. IEEE Computer Society, 2003, pp. 726–736.

APÉNDICE

REFERENCIAS DE ESTUDIOS SELECCIONADOS

[A1] M. Cremaschi and F. De Paoli, "A practical approach to services composition through light semantic descriptions," in Lecture Notes in Computer Science, vol. 11116 LNCS, , 2018, pp. 130–145.

[A2] Cremaschi and F. De Paoli, "Toward automatic semantic API descriptions to support services composition," in Lecture Notes in Computer Science, vol. 10465 LNCS, 2017, pp. 159–167.

[A3] H. Ed-Douibi, J. L. Canovas Izquierdo, F. Bordeleau, and J. Cabot, "WAPIml: Towards a modeling infrastructure for web APIs," in Proceedings - 2019 ACM/IEEE 22nd International Conference on Model Driven Engineering Languages and Systems Companion, 2019, pp. 748–752, doi: 10.1109/MODELS-C.2019.00116.

[A4] H. Ed-Douibi, J. L. Cánovas Izquierdo, and J. Cabot, "APIComposer: Data-driven composition of REST APIs," in Lecture Notes in Computer Science, vol. 11116 LNCS, 2018, pp. 161–169.

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 28,2026 at 01:31:12 UTC from IEEE Xplore. Restrictions apply.

[A5] H. Ed-Douibi, G. Daniel, and J. Cabot, "Openapi BOT: A chatbot to help you understand rest APIS," in Lecture Notes in Computer Science, vol. 12128 LNCS, 2020, pp. 538–542.

[A6] H. Ed-douibi, J. L. C. Izquierdo, and J. Cabot, "Example-driven web API specification discovery," in Lecture Notes in Computer Science, vol. 10376 LNCS, 2017, pp. 267–284.

[A7] B. Pirelli, N. Nessler, and A. Wegmann, "Co-design of business and it services - a tool-supported approach," in Lecture Notes in Computer Science, vol. 11434 LNCS, 2019, pp. 27–40.

[A8] M. Vaziri, L. Mandel, A. Shinnar, J. Siméon, and M. Hirzel, "Generating Chat Bots from Web API Specifications," in Proceedings of the 2017 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software, 2017, pp. 44–57, doi: 10.1145/3133850.3133864.

[A9] J. Yang, E. Wittern, A. T. T. Ying, J. Dolby, and L. Tan, "Towards Extracting Web API Specifications from Documentation," in Proceedings of the 15th International Conference on Mining Software Repositories, 2018, pp. 454–464, doi: 10.1145/3196398.3196411.

[A10] P. Espinoza-Arias, D. Garijo, and O. Corcho, "Mapping the Web Ontology Language to the OpenAPI Specification," in Lecture Notes in Computer Science, vol. 12584 LNCS, 2020, pp. 117–127.

[A11] A. Gamez-Diaz, P. Fernandez, and A. Ruiz-Cortes, "Automating SLA-Driven API Development with SLA4OAI," in Lecture Notes in Computer Science, vol. 11895 LNCS, 2019, pp. 20–35.

[A12] C. González-Mora, C. Barros, I. Garrigós, J. Zubcoff, E. Lloret, and J. N. Mazón, "Applying natural language processing techniques to generate open data web apis documentation," in Lecture Notes in Computer Science, vol. 12128 LNCS, 2020, pp. 416–432.

[A13] A. Karavisileiou, N. Mainas, and E. G. M. Petrakis, "Ontology for OpenAPI REST Services Descriptions," in Proceedings - International Conference on Tools with Artificial Intelligence, ICTAI, 2020, vol. 2020-November, pp. 35–40, doi: 10.1109/ICTAI50040.2020.00016.

[A14] F. Khodadadi and R. O. Sinnott, "A Semantic-aware Framework for Service Definition and Discovery in the Internet of Things Using CoAP," Procedia Comput. Sci., vol. 113, pp. 146–153, 2017, doi: 10.1016/j.procs.2017.08.334.

[A15] S. P. Ma, M. J. Hsu, H. J. Chen, and Y. S. Su, "API Prober – A Tool for Analyzing Web API Features and Clustering Web APIs," in Lecture Notes on Data Engineering and Communications Technologies, vol. 41, 2020, pp. 81–96.

[A16] I. Koren and R. Klamma, "The Exploitation of OpenAPI Documentation for the Generation of Web Frontends," in Companion Proceedings of the The Web Conference 2018, 2018, pp. 781–787, doi: 10.1145/3184558.3188740.

[A17] M. N. Lucky, M. Cremaschi, B. Lodigiani, A. Menolascina, and F. De Paoli, "Enriching API descriptions by adding API profiles through semantic annotation," in Lecture Notes in Computer Science, vol. 9936 LNCS, 2016, pp. 780–794.

[A18] A. Martin-Lopez, "Automated Analysis of Inter-Parameter Dependencies in Web APIs," in Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Companion Proceedings, 2020, pp. 140–142, doi: 10.1145/3377812.3382173.

[A19] F. A. Musyaffa, L. Halilaj, R. Siebes, F. Orlandi, and S. Auer, "Minimally invasive semantification of light weight service descriptions," in Proceedings - 2016 IEEE International Conference on Web Services, 2016, pp. 672–677, doi: 10.1109/ICWS.2016.93.

[A20] A. Netedu, S. C. Buraga, P. Diac, and L. Ţucăr, "A Web Service Composition Method Based on OpenAPI Semantic Annotations," in Lecture Notes on Data Engineering and Communications Technologies, vol. 41, 2020, pp. 342–357.

[A21] N. Oostvogels, J. De Koster, and W. De Meuter, "Inter-parameter constraints in contemporary web APIs," in Lecture Notes in Computer Science, vol. 10360 LNCS, 2017, pp. 323–335.

[A22] A. Martin-Lopez, S. Segura, and A. Ruiz-Cortés, "RESTest: Black-box constraint-based testing of RESTful web APIs," in Lecture Notes in Computer Science, vol. 12571 LNCS, 2020, pp. 459–475.

[A23] H. Ed-Douibi, J. L. Canovas Izquierdo, and J. Cabot, "Automatic generation of test cases for REST APIs: A specification-based approach," in Proceedings - 2018 IEEE 22nd International Enterprise Distributed Object Computing Conference, EDOC 2018, 2018, pp. 181–190, doi: 10.1109/EDOC.2018.00031.

[A24] V. Atlidakis, P. Godefroid, and M. Polishchuk, "RESTler: Stateful REST API Fuzzing," in Proceedings of the 41st International Conference on Software Engineering, 2019, pp. 748–758, doi: 10.1109/ICSE.2019.00083.

[A25] Y. Tashtoush, M. N. Alrashdan, O. Salameh, and M. Alsmirat, "Swagger-based jQuery Ajax validation," in 2019 IEEE 9th Annual Computing and Communication Workshop and Conference, 2019, pp. 69–72, doi: 10.1109/CCWC.2019.8666542.

[A26] E. Viglianisi, M. Dallago, and M. Ceccato, "RESTTESTGEN: Automated Black-Box Testing of RESTful APIs," in Proceedings - 2020 IEEE 13th International Conference on Software Testing, Verification and Validation, 2020, pp. 142–152, doi: 10.1109/ICST46399.2020.00024.

[A27] J. Wang, X. Bai, L. Li, Z. Ji, and H. Ma, "A Model-Based Framework for Cloud API Testing," in 2017 IEEE 41st Annual Computer Software and Applications Conference (COMPSAC), 2017, vol. 2, pp. 60–65, doi: 10.1109/COMPSAC.2017.24.

[A28] E. Wittern, A. T. T. Ying, Y. Zheng, J. Dolby, and J. A. Laredo, "Statically Checking Web API Requests in JavaScript," in Proceedings of the 39th International Conference on Software Engineering, 2017, pp. 244–254, doi: 10.1109/ICSE.2017.30.

[A29] L. Baresi, M. Garriga, and A. De Renzis, "Microservices identification through interface analysis," in Lecture Notes in Computer Science, vol. 10465 LNCS, 2017, pp. 19–33.

[A30] J. Bogner, S. Wagner, and A. Zimmermann, "Collecting service-based maintainability metrics from restful api descriptions: Static analysis and threshold derivation," in Communications in Computer and Information Science, vol. 1269 CCIS, 2020, pp. 215–227.

[A31] V. A. Cunha et al., "A Network Service for Preventing Data Leakage from IoT Cloud-assisted Equipment," in Proceedings - IEEE Symposium on Computers and Communications, 2019, vol. 2019-June, pp. 1–7, doi: 10.1109/ISCC47284.2019.8969719.

[A32] A. Gamez-Diaz, P. Fernandez, and A. Ruiz-Cortés, "Governify for APIs: SLA-Driven Ecosystem for API Governance," in Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, 2019, pp. 1120–1123, doi: 10.1145/3338906.3341176.

[A33] D. Garijo and M. Osorio, "OBA: An Ontology-Based Framework for Creating REST APIs for Knowledge Graphs," in Lecture Notes in Computer Science, vol. 12507 LNCS, 2020, pp. 48–64.

[A34] M. Grambow, E. Wittern, and D. Bermbach, "Benchmarking the Performance of Microservice Applications," SIGAPP Appl. Comput. Rev., vol. 20, no. 3, pp. 20–34, 2020, doi: 10.1145/3429204.3429206.

[A35] O. Hylli, A. Ruokonen, N. Mäkitalo, and K. Systä, "Orchestrating the Internet of Things Dynamically: Research Paper," 2016, doi: 10.1145/3007203.3007216.

[A36] B. A. Sanchez, K. Barmpis, P. Neubauer, R. F. Paige, and D. S. Kolovos, "Restmule: Enabling Resilient Clients for Remote APIs," in Proceedings of the 15th International Conference on Mining Software Repositories, 2018, pp. 537–541, doi: 10.1145/3196398.3196405.

[A37] D. Sferruzza, J. Rocheteau, C. Attiogbé, and A. Lanoix, "A model-driven method for fast building consistent web services from OpenAPI-compatible models," in Communications in Computer and Information Science, vol. 991, Springer International Publishing, 2019, pp. 9–33.

[A38] J. Suhonen, O. Kivelä, T. Laukkarinen, and M. Hännikäinen, "Unified Service Access for Wireless Sensor Networks," in Proceedings of the Third International Workshop on Software Engineering for Sensor Network Applications, 2012, pp. 49–55.

[A39] L. Sun, S. Chen, Q. Liang, and Z. Feng, "A QoS-based self-adaptive framework for OpenAPI," in Proceedings - 2011 7th International Conference on Computational Intelligence and Security, 2011, pp. 204–208, doi: 10.1109/CIS.2011.53.

[A40] E. Wittern, A. Cha, and J. A. Laredo, "Generating GraphQL-wrappers for REST(-like) APIs," in Lecture Notes in Computer Science, vol. 10845 LNCS, 2018, pp. 65–83.

[A41] S. Kotstein and C. Decker, "Navigational Support for Non HATEOAS-Compliant Web-Based APIs," in Communications in Computer and Information Science, vol. 1310, 2020, pp. 169–188.

[A42] S. Laso, M. Linaje, J. Garcia-Alonso, J. M. Murillo, and J. Berrocal, "Deployment of APIs on Android Mobile Devices and Microcontrollers," in 2020 IEEE International Conference on Pervasive Computing and Communications Workshops, 2020, pp. 1–3, doi: 10.1109/PerComWorkshops48775.2020.9156208.

[A43] D. Lee, S. Jeong, and J. W. K. Hong, "OpenAPI-based message router for mashup service development," in 19th Asia-Pacific Network Operations and Management Symposium: Managing a World of Things, 2017, pp. 94–99, doi: 10.1109/APNOMS.2017.8094185.

[A44] S. P. Ma, H. J. Lin, C. W. Lan, W. T. Lee, and M. J. Hsu, "Real-World RESTful Service Composition: A Transformation-Annotation-Discovery Approach," in Proceedings - 2017 IEEE 10th International Conference on Service-Oriented Computing and Applications, 2017, vol. 2017-January, pp. 1–8, doi: 10.1109/SOCA.2017.9.

[A45] A. Meroño-Peñuela and R. Hoekstra, "Automatic query-centric API for routine access to linked data," in Lecture Notes in Computer Science, vol. 10588 LNCS, 2017, pp. 334–349.

[A46] M. Noura, S. Heil, and M. Gaedke, "Webifying heterogenous internet of things devices," in Lecture Notes in Computer Science, vol. 11496 LNCS, 2019, pp. 509–513.

[A47] F. Haupt, F. Leymann, A. Scherer, and K. Vukojevic-Haupt, "A Framework for the Structural Analysis of REST APIs," in Proceedings - 2017 IEEE International Conference on Software Architecture, 2017, pp. 55–58, doi: 10.1109/ICSA.2017.40.

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on May 28,2026 at 01:31:12 UTC from IEEE Xplore. Restrictions apply.